Submission Requirements

Project Category: B1AI DEVOP1
Author: Roni Fernandes Dias / François LANGE
Date: June 24, 2026

Table of Contents

Project Submission Requirements

Parameter	Details
Institution	Brevet de Technicien Supérieur
Module	DEVOP1 - Project Submission Requirements
Date	24.06.2026
Delivery	24.06.2026
Presentation	01.07.2026
Room	SC-27
Candidates	Roni Fernandes Dias / François LANGE

Part 1 Implementing Agile Methods

Agile Project Management Implementation
Design and implement a collaborative project management system to enhance visibility, agility, and coordination between technical and non-technical team members. The chosen platform should integrate smoothly with existing DevOps workflows and support essential agile practices, including task tracking, sprint planning, progress monitoring, and team collaboration.
Tool Selection Options
You are required to evaluate and select one of the following tools based on factors such as project requirements, integration capabilities, and licensing considerations:
Jira Software
Microsoft Planner
Taiga
Trello
Asana
Expected Deliverables
A fully configured project management platform featuring:
Custom boards and workflows tailored to your teams methodology

Clear assignment of individual workloads and grouping of related tasks
Integration with your CI/CD pipelines
Direct linkage to your Git repository for code tracking
A dashboard that enables real-time monitoring of progress, sprint status, and backlog health
Ongoing visibility of team activity for the supervising teacher
Weekly progress presentations to your teacher, demonstrating what has been accomplished and what's planned next

Part 2 Set up version control and repository structure.

Implement Git as a version control system to enable efficient source code management, parallel development, and streamlined collaboration across teams. This project will demonstrate core Git functionalities and integrate Git workflows with remote repositories (e.g., GitHub, GitLab) to support a modern DevOps pipeline.
This project introduces and applies key Git commands and workflows in a collaborative software development environment. The focus is on enabling developers to:
Create local repositories from remote sources
Track and manage code changes
Work independently using branches
Merge contributions into shared codebases
Synchronize work using remote repositories
The project will also emphasize best practices in commit hygiene, branching strategies (e.g., Git Flow or GitHub Flow), and team collaboration through Git platforms like GitHub or GitLab.
Expected Deliverables:
A fully initialized Git repository with:
At least 3 active feature branches
Clean and annotated commit history
Completed merge workflows using pull/merge requests
Integration of a CI/CD workflow with GitHub Actions or GitLab CI
Markdown-based documentation (README.md) covering:
Git command usage
Workflow standards
Branching strategy
A Git cheat sheet for developers (basic to advanced)

Part 3 Setup a Virtual Environment for Developer Operations

Implement a scalable, flexible, and efficient virtualization infrastructure on Linux using DevOps methodologies to ensure continuous integration, automated deployment, infrastructure as code (IaC), performance monitoring.
This project will guide the implementation of Docker-based containerization for a modular application stack (e.g., a web frontend, backend API, and a database). It will include:
Docker: To create and manage lightweight, isolated containers that package software and its dependencies.
Dockerfile: To define reproducible builds for each component of the application.
Docker Compose: To orchestrate multiple containers and define service dependencies in a single YAML file.

The containerized environment will allow developers to work in isolated environments, ensure consistent builds across systems, and simplify the deployment pipeline in CI/CD workflows.

Task 1 - Docker Installation & Setup

Install Docker Engine on Linux or Windows.
Configure Docker daemon and test with sample containers (hello-world).

Task 2 - Creating Dockerfiles

Write Dockerfiles for each service (e.g., Node.js/Flask backend, React frontend, PostgreSQL/MySQL database).
Apply best practices: small base images (e.g., alpine), multi-stage builds, proper use of CMD, ENTRYPOINT, COPY, RUN, EXPOSE, and caching layers.

Task 3 - Building and Running Containers

Use docker build, docker run, docker exec, and docker logs to manage container lifecycles.
Tag images and store them in Docker Hub or a private registry.

Task 4 - Creating Docker Compose Configuration

Define services, volumes, networks, and environment variables in docker-compose.yml.
Use docker-compose up, down, logs, build, and exec for orchestration.

Task 5 - CI/CD Integration

Automate image builds and deployments using GitHub Actions or GitLab CI.

Project Goals:
Containerize all application components using Docker
Create clean, maintainable Dockerfiles for each service
Use Docker Compose to manage multi-container applications
Enable environment portability across dev, staging, and prod
Integrate image build/test/deploy steps into a CI/CD pipeline
Expected Deliverables:
Working Dockerfile for each service
Complete docker-compose.yml with multi-service orchestration
Scripts for starting, stopping, and rebuilding the stack
Sample logs from docker logs or docker-compose logs
CI/CD pipeline configuration for automatic builds

Part 4 Deploy a containerized MySQL Database

This project focuses on leveraging Docker to deploy a MySQL database in a reproducible and isolated environment. The goal is to design a schema that aligns with the structure of pre-processed data, and to use Python's SQLAlchemy library to programmatically populate and validate the database.
By using Docker, we eliminate manual installation and configuration steps, enabling fast and portable setup across different development or production environments.

Task 1: Containerize MySQL with Docker

Create a docker-compose.yml file to spin up a MySQL container with environment variables (root password, database name, user credentials).
Configure volume mounts for persistent data storage.

Task 2: Define and create the MySQL Database

Use SQL scripts or SQLAlchemy ORM models to define the database and initialize it automatically via Docker entrypoint or Python script.

Task 3: Design and Apply Table Schemas

Define tables in alignment with the processed dataset using SQLAlchemy models or raw SQL scripts.
Ensure data types and relationships are clearly specified.

Task 4: Data Loading with SQLAlchemy

Use a Python script with SQLAlchemy to connect to the running MySQL container and load transformed data.
Handle connection parameters via environment variables or .env file shared with Docker Compose.

Use one of your groups DATOP1 projects to simplify your transition from your previous project onto Docker.

Part 5 Deploy a containerized web application

To develop, containerize, and deploy a web application using Docker, ensuring a consistent and portable environment across development, testing, and production stages. The goal is to simplify application management, enable scalability, and integrate with modern CI/CD workflows.
This project focuses on building a modular web application (e.g., using Flask, Node.js, or Django for the backend and React or plain HTML/CSS for the frontend), packaging it with Docker, and orchestrating it using Docker Compose.
The application will be fully containerized to ensure that all dependencies, configurations, and environments are self-contained. This enables seamless deployment on any system with Docker, facilitates horizontal scaling, and supports DevOps practices like automated testing and continuous delivery.

Task 1 - Develop the Web Application

Build a basic web application with a frontend and backend component.

Task 2 - Write a Dockerfile

Create a Dockerfile to containerize the backend application:
Set the base image (e.g., python:3.11-slim or node:18)
Copy source code, install dependencies
Expose ports and define CMD or ENTRYPOINT

Task 3 - Add a Frontend or Static UI

Optionally include a separate Dockerfile for the frontend (e.g., built with React).
embed the frontend in the backend container.

Task 4 - Use Docker Compose for Orchestration

Create a docker-compose.yml file to define and run:
Web service container
Optional database (e.g., PostgreSQL, MySQL, or MongoDB)
Define networks and volumes for service communication and data persistence.

Task 5 - Configure Environment Variables

Use a .env file to manage secrets and environment-specific variables.

Task 6 - Build, Run, and Test Containers

Run services with docker-compose up
Verify application functionality via browser or Postman
Expected Deliverables:
A functional web application containerized with Docker
A Dockerfile for the backend (and frontend if applicable)
A docker-compose.yml orchestrating all services
.env file for dynamic configuration
Docker volumes for persistent storage (e.g., database)
Application accessible at http://localhost:

Use one of your groups DATOP1 projects to simplify your transition from your previous project onto Docker.

Updated Part 6 Deploy a containerized Real-Time Monitoring and Visualization with Zabbix
This project focuses on building a Dockerized Zabbix stack that integrates with real-time data sources to monitor system health, performance trends, and resource usage. All services will be containerized and

managed using Docker Compose to ensure environment consistency, fast deployment, and simplified configuration. Zabbix will serve as the central platform for data collection, interactive visualization, and alerting administrators of abnormal behavior.
Expected Deliverables

Task 1 Containerize Zabbix and Components

A complete docker-compose.yml file defining services for:
Zabbix Server
Zabbix Web Interface
Global MySQL Database: A shared database instance to be utilized by the Zabbix stack for all persistent storage.
Docker volumes configured for persistent storage of:
Zabbix configuration files
Zabbix database metrics data

Task 2 Integrate Data Collection

Provisioning of Zabbix agents (or SNMP/Active checks) for system-level metrics collection.
Verified data source connections in the Zabbix interface.

Task 3 Design Dashboards

At least three Zabbix dashboards that display:
System resource metrics (CPU, memory, disk, network)
Application-specific custom metrics
Database performance indicators
Management Dashboard: A centralized "Control Panel" dashboard created to provide an at-a-glance view of the entire monitored ecosystem, including service status and alert summaries.
Dashboard configurations exported as templates/XML files and versioned in Git.

Task 4 Set Up Alerts and Notifications

Alerting rules configured within Zabbix triggers and action policies.
Three notification channels set up and tested to propagate alarms:
Email
Discord
Microsoft Teams
Triggered test alerts demonstrating notification delivery.

Task 5 Enable Real-Time Monitoring

Real-time metrics displayed in Zabbix with appropriate refresh intervals.
Configured thresholds and visual indicators for performance status.

Use one of your groups DATOP1 projects to simplify your transition from your previous project onto Docker.

Part 7 Containerized Jenkins and Kubernetes

Deploy and integrate Jenkins as a continuous integration/continuous delivery (CI/CD) automation server within a Kubernetes cluster. This project aims to enable scalable, automated build, test, and deployment pipelines for modern cloud-native applications.
This project will involve the containerized deployment of Jenkins on a Kubernetes cluster and the configuration of CI/CD pipelines that build, test, and deploy applications automatically. Jenkins will run as a pod within the cluster and dynamically spin up build agents using Kubernetes as the execution backend.
Key outcomes include infrastructure-as-code (IaC) integration, automated workflows from Git commit to production deployment, and support for modern development patterns (microservices, Helm, Dockerized apps, etc.).

Task 1 - Kubernetes Cluster Setup

Provision a local (Minikube, Kind) or cloud-managed Kubernetes cluster (GKE, EKS, AKS).
Install basic tools: kubectl, helm, and configure namespaces, role bindings, and persistent storage.

Task 2 - Deploy Jenkins on Kubernetes

Install Jenkins

Task 3 - Configure Jenkins Master and Agents

Set up Jenkins master to use Kubernetes plugin.
Define pod templates for dynamic agent provisioning.
Configure Jenkins to use Docker-in-Docker for image builds (if needed).

Task 4 - CI/CD Pipeline Design

Create Jenkins pipelines (declarative Jenkinsfile) for:
Pulling code from Git (GitHub, GitLab)
Running unit and integration tests
Building Docker images
Pushing images to Docker Hub or private registry
Deploying to Kubernetes via kubectl or Helm

Expected Deliverables:
Kubernetes Infrastructure:
Running Kubernetes cluster with Jenkins
Jenkins CI/CD Pipelines:
Jenkinsfile(s) stored in Git repository

Pipeline stages for build, test, Docker packaging, and Kubernetes deployment
Container Image & Deployment:
Dockerfile and Kubernetes deployment manifests
Application container built, tagged, and deployed to the cluster via CI/CD

Part 8 Project delivery

Ensure that all project tasks and activities outlined in the project plan are fully completed and meet the expected quality standards and functional requirements. Maintain regular communication with your supervisor throughout all phases of the project to ensure alignment and effective implementation of each component.
Before submission, conduct thorough testing of all deliverables to identify and resolve any bugs or issues.
Prepare and validate the necessary infrastructure, environments, and configurations required to support deployment.
You must submit a complete and accessible project repositoryincluding the test environment, source code, configuration files, and documentationno later than June 24, 2026, to allow the teacher to review and evaluate your work effectively.

Part 9 Presentation

Conduct a project closure meeting to review the projects performance, achievements, and lessons learned.
Acknowledge individual achievements and express appreciation for dedication and hard work.
The precise schedule is set for the 2nd of July 2025. The group is allotted a 30-minute time slot to present and defend their project. Within this timeframe, each candidate should have presented his/her part of the workload during the project for the presentation and 5-10 minutes to respond to questions.
Presentation Guidelines
During your project presentation, ensure you cover the following key aspects:
Project Timeline & Progress
Present the development progress throughout the project lifecycle, highlighting major milestones and phases.
Hosting Environment
Demonstrate where your project is currently hosted (e.g., cloud platform, local server, Kubernetes cluster).
Source Code Repository
Share and walk through your project's Git repository, showcasing structure, branches, and any relevant CI/CD integrations.
Infrastructure Overview
Provide a high-level overview of your infrastructure, including diagrams or tools (e.g., Docker Compose, Kubernetes manifests).
Containerized Components
Clearly identify and explain each containerized service in your stack, including:
Zabbix for monitoring and visualization
MySQL for data storage
Web application frontend/backend services
Jenkins for CI/CD automation

Kubernetes as the orchestration layer
Demonstration of Components
Live-demo or provide screenshots of:
Zabbix dashboards
MySQL database instance or queries
The running web application
Jenkins pipelines
Kubernetes pods, services, and deployments

Part 10 Evaluation

Part 1: Technical Implementation (20 points)

Covers Agile tooling, Git, Docker, Jenkins, Kubernetes

Part 2: Individual Workload (20 points)

Evaluates each student's personal contribution, ownership, and execution

Part 3: Presentation & Documentation (20 points)

Grades how the project is presented and how well its documented
Each of the three parts is out of 20 points Final Grade = (Part 1 + Part 2 + Part 3) ÷ 3

Evaluation Criteria

Criteria	Points
1. Agile Project Management Setup (Trello, Jira, etc.)	4
2. Git Usage and Workflow (branches, commits, CI links)	6
3. Dockerization of All Components (Dockerfiles, Compose)	6
4. Jenkins CI/CD Pipelines (pull, test, build, deploy)	2
5. Kubernetes Integration (pods, services, manifests)	2

Total	**20**

Evaluation Criteria

Criteria	Points
1. Clear ownership of at least one project module	5
2. Git contribution (commits, merges, documented code)	5
3. Quality and completeness of the assigned task	5
4. Teamwork and collaboration (responsiveness, support)	5
Total	**20**

Evaluation Criteria

Criteria	Points
1. Presentation of project flow, architecture, and outcomes	5
2. Live demo or screenshots of working services (Zabbix, app, DB, etc.)	5
3. Technical documentation (README, config files, usage guide)	5
4. Clarity, structure, and engagement during oral presentation	5
Total	**20**

Appendix: Additional Project Documentation

The following PDF documents are generated and available in the project repository:

- description.pdf
- domainproject.pdf
- Project.pdf
- Retro_Planning.pdf
- persona-template.pdf
- User stories.pdf
- Wireframes.pdf
- architecture.pdf
- Backup_Procedure.pdf
- Data_Ingestion.pdf
- disaster_recovery_plan.pdf
- distributed_deployment.pdf
- docker_connection.pdf
- Env_Configuration.pdf
- Final_Report.pdf
- Global_Index.pdf
- Header_Footer_Antigravity.pdf
- Header_Footer_Gemini.pdf
- Installation_Guide.pdf
- Operator_Installation_Guide.pdf
- Presentation_Technical.pdf
- Presentation_User.pdf
- project_report.pdf
- Recommendations.pdf
- retro_planning.pdf
- Scrum_Artifacts.pdf
- Start_Stop_Procedures.pdf
- taiga_audit_report.pdf
- Technical_Document.pdf
- Uninstall_Guide.pdf
- URL_Formats.pdf

- User_Description.pdf
- User_Guide.pdf
- Wiki_Home.pdf
- WSL_Deployment.pdf
- Virtualbox_Deployment.pdf
- Hyper-V_Deployment.pdf
- zabbix_monitoring.pdf
- Logs_information.pdf
- How_to_change_webhooks_and_emails.pdf
- Readme.pdf

Page Search Index